day1

Django REST Framework `Serializer` use করলে কোন কোন method override করা লাগে / করা যায় — একদম basics থেকে clean করে বলছি।

🔹 1️⃣ `Serializer` use করলে কেন method override লাগে?

`serializers.Serializer` **database জানে না।**
তাই **data save / update করার জন্য নিজে method লিখতে হয়।**

সবচেয়ে important ২টা method:

- `create()`
- `update()`

🔹 2️⃣ `create()` method (NEW object বানানোর জন্য)

👉 যখন `serializer.save()` call করো **POST request এ**

```
class EmployeeSerializer(serializers.Serializer):
    name = serializers.CharField(max_length=100)
    age = serializers.IntegerField()
    salary = serializers.FloatField()

    def create(self, validated_data):
        return Employee.objects.create(**validated_data)
```

📌 Flow:

```
serializer = EmployeeSerializer(data=request.data)
serializer.is_valid()
serializer.save()   # 👉 create() call হয়
```

🔹 3️⃣ `update()` method (OLD object update করার জন্য)

👉 যখন PUT / PATCH request আসে

```
class EmployeeSerializer(serializers.Serializer):
    name = serializers.CharField(max_length=100)
    age = serializers.IntegerField()
    salary = serializers.FloatField()

    def update(self, instance, validated_data):
        instance.name = validated_data.get("name", instance.name)
        instance.age = validated_data.get("age", instance.age)
        instance.salary = validated_data.get("salary", instance.salary)
        instance.save()
        return instance
```

📌 Flow:

```
serializer = EmployeeSerializer(instance=emp, data=request.data)
serializer.is_valid()
serializer.save()   # 👉 update() call হয়
```

🔹 4️⃣ Validation override (খুব important)

✅ Field-level validation

```
def validate_age(self, value):
    if value < 18:
        raise serializers.ValidationError("Age must be 18+")
    return value
```

✅ Object-level validation (multiple field check)

```
def validate(self, data):
    if data["age"] < 25 and data["salary"] > 50000:
        raise serializers.ValidationError(
            "Young employee cannot have this salary"
        )
    return data
```

🔹 5️⃣ `to_representation()` override (Output customize করতে)

👉 Response data change করতে চাইলে

```
def to_representation(self, instance):
    data = super().to_representation(instance)
    data["salary"] = f"${data['salary']}"
    return data
```

🔹 6️⃣ `to_internal_value()` override (Input modify করতে)

👉 Input data আগে modify করতে চাইলে

```
def to_internal_value(self, data):
    data["name"] = data.get("name", "").title()
    return super().to_internal_value(data)
```

🔹 7️⃣ Summary (Exam / Interview friendly)

Method	কেন ব্যবহার হয়
`create()`	নতুন object create
`update()`	existing object update
`validate_<field>()`	single field validation
`validate()`	multiple field validation
`to_representation()`	response format change
`to_internal_value()`	input preprocess

🔥 Tip (Real world)

👉 যদি model থাকে, **৯০% সময় `ModelSerializer` use করো**
👉 `Serializer` use করো যখন:

- External API data
- Non-model data
- Custom logic

Nice question 👍
চল **DELETE আর PATCH** দুটোই **simple বাংলায় + example** দিয়ে বুঝি।

🔴 DELETE (data delete করার জন্য)

📌 Serializer এ DELETE method থাকে না ❌

DELETE **serializer দিয়ে না**,
👉 **View / ViewSet** থেকে handle করা হয়।

Example (APIView)

```
class EmployeeDelete(APIView):
    def delete(self, request, pk):
        emp = Employee.objects.get(id=pk)
        emp.delete()
        return Response(
            {"message": "Employee deleted"},
            status=status.HTTP_204_NO_CONTENT
        )
```

👉 এখানে:

- `serializer.save()` ❌
- `create()` / `update()` ❌

🟡 PATCH (partial update)

👉 PATCH মানে **কিছু field update**, সব না।

Serializer side এ কি হয়?

PATCH এ:

- `update()` method call হয় ✅
- কিন্তু `partial=True` দিতে হয়

Example

```
class EmployeeUpdate(APIView):
    def patch(self, request, pk):
        emp = Employee.objects.get(id=pk)
        serializer = EmployeeSerializer(
```

```
            emp,
            data=request.data,
            partial=True    # ⭐ IMPORTANT
        )
        serializer.is_valid(raise_exception=True)
        serializer.save()    # 👉 update() call হবে
        return Response(serializer.data)
```

🔁 PUT vs PATCH (1 line difference)

PUT	PATCH
সব field দিতে হয়	শুধু যেগুলো change
`partial=False` (default)	`partial=True`
Full replace	Partial update

🔥 PATCH এ validation কিভাবে কাজ করে?

```
def update(self, instance, validated_data):
    for attr, value in validated_data.items():
        setattr(instance, attr, value)
    instance.save()
    return instance
```

👉 PATCH হলে `validated_data` তে **শুধু আসা field গুলোই থাকবে**

⚠️ Common mistake

❌ PATCH এ `partial=True` না দেয়া
→ serializer error দেয়:
`"This field is required"`

🧠 Quick summary (Interview ready)

- ❌ DELETE → Serializer লাগে না
- ✅ PATCH → `update()` + `partial=True`
- `serializer.save()`

- POST → `create()`
- PUT / PATCH → `update()`

ভালো প্রশ্ন 🔥
Django REST Framework `APIView` এর ভেতরে কোন কোন method থাকে — clean table + example দিচ্ছি।

🔹 APIView এ HTTP Method গুলো

👉 `APIView` এ আমরা **HTTP verb অনুযায়ী method লিখি**:

HTTP Method	APIView method
GET	`get()`
POST	`post()`
PUT	`put()`
PATCH	`patch()`
DELETE	`delete()`
HEAD	`head()`
OPTIONS	`options()`

👉 Real world এ বেশি use হয়:
GET, POST, PUT, PATCH, DELETE

🔹 Example: Full CRUD APIView

```
class EmployeeAPIView(APIView):

    def get(self, request, pk=None):
        if pk:
            emp = Employee.objects.get(id=pk)
            serializer = EmployeeSerializer(emp)
        else:
            emp = Employee.objects.all()
            serializer = EmployeeSerializer(emp, many=True)
        return Response(serializer.data)

    def post(self, request):
```

```
        serializer = EmployeeSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        serializer.save()
        return Response(serializer.data, status=201)

    def put(self, request, pk):
        emp = Employee.objects.get(id=pk)
        serializer = EmployeeSerializer(emp, data=request.data)
        serializer.is_valid(raise_exception=True)
        serializer.save()
        return Response(serializer.data)

    def patch(self, request, pk):
        emp = Employee.objects.get(id=pk)
        serializer = EmployeeSerializer(
            emp, data=request.data, partial=True
        )
        serializer.is_valid(raise_exception=True)
        serializer.save()
        return Response(serializer.data)

    def delete(self, request, pk):
        emp = Employee.objects.get(id=pk)
        emp.delete()
        return Response(status=204)
```

🔹 APIView এর built-in useful attributes

APIView শুধু method না, কিছু **important features** দেয়:

```
class EmployeeAPIView(APIView):
    permission_classes = [IsAuthenticated]
    authentication_classes = [TokenAuthentication]
```

Common attributes:

- `permission_classes`
- `authentication_classes`
- `throttle_classes`
- `parser_classes`
- `renderer_classes`

🔹 APIView vs ViewSet (short idea)

APIView	ViewSet
Manual control	Less code
Beginner friendly	Production friendly
Explicit methods	Router based

🧠 Interview short answer

> APIView এ HTTP method অনুযায়ী `get, post, put, patch, delete` define করা হয়।

Short answer: **না—সব method এভাবে লিখতেই হবে না।**
এখন clean করে explain করি 👇

🔹 1️⃣ APIView এ method "এইভাবেই" হয় কেন?

`APIView` **HTTP verb–based** class, তাই:

- `GET → get()`
- `POST → post()`
- `PUT → put()`
- `PATCH → patch()`
- `DELETE → delete()`

👉 **এগুলোর নাম change করা যাবে না**
👉 এভাবে না লিখলে DRF call করবে না

✅ তাই **APIView হলে এই pattern follow করতেই হয়**

🔹 2️⃣ কিন্তু… সব method লিখতে হবে? ❌

❌ না।
শুধু **যেগুলো দরকার**, সেগুলোই লেখো।

Example: Only GET + POST

```
class EmployeeAPIView(APIView):
```

```
    def get(self, request):
        ...

    def post(self, request):
        ...
```

👉 PUT / PATCH / DELETE না লিখলেও problem নেই

🔹 3️⃣ যদি সব নিজে না লিখতে চাই?

তখন **higher-level class** use করো 👇

🔹 4️⃣ GenericAPIView + Mixins (half auto)

```
from rest_framework.generics import GenericAPIView
from rest_framework.mixins import (
    ListModelMixin, CreateModelMixin,
    UpdateModelMixin, DestroyModelMixin
)

class EmployeeAPI(
    GenericAPIView,
    ListModelMixin,
    CreateModelMixin,
    UpdateModelMixin,
    DestroyModelMixin
):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
```

👉 এখানে তোমাকে:

- `get()`
- `post()`
- `put()`
- `delete()`

লিখতে হয়নি 😎

🔹 5️⃣ সবচেয়ে সহজ: ViewSet / ModelViewSet

```
from rest_framework.viewsets import ModelViewSet

class EmployeeViewSet(ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
```

👉 Automatically:

- list (GET)
- retrieve (GET)
- create (POST)
- update (PUT)
- partial_update (PATCH)
- destroy (DELETE)

🔹 6️⃣ কোনটা কখন use করবে?

Need	Use
Full control	APIView
Less code + control	GenericAPIView
CRUD fast	ModelViewSet

🧠 Final clear answer

- ✅ APIView → method গুলো **এই নামেই** হবে
- ❌ সব method লিখতে হবে না
- ✅ Alternative আছে (GenericAPIView / ViewSet)

খুব ভালো প্রশ্ন 👍
এটা বুঝে গেলে **APIView পুরো clear** হয়ে যাবে।

🔹 Short answer

👉 **হ্যাঁ, APIView use করলে operation অনুযায়ী method লিখতে হবে।**
👉 **না লিখলে ওই operation কাজ করবে না।**

🔹 Detail explanation (simple ভাষায়)

`APIView` HTTP request আসলে **method name দেখে** decide করে কোন code চলবে।

Example

```
class EmployeeAPIView(APIView):
    pass
```

এখন যদি:

- `GET /employees/` পাঠাও
  ❌ Error আসবে:

```
405 Method Not Allowed
```

👉 কারণ `get()` method লেখা নাই

🔹 Case by case বুঝি

❌ GET লিখো নাই

```
# request: GET
405 Method Not Allowed
```

❌ POST লিখো নাই

```
# request: POST
405 Method Not Allowed
```

👉 DRF নিজে কিছু করে না,
তুমি যেটা লিখো, সেটাই চালায়

🔹 Partial example

```
class EmployeeAPIView(APIView):

    def get(self, request):
        return Response({"msg": "GET works"})
```

Request	Result
GET	✅ 200 OK
POST	❌ 405
PUT	❌ 405
DELETE	❌ 405

🔹 Why this happens (important concept)

`APIView` internally করে:

```
if request.method == "GET":
    call get()
elif request.method == "POST":
    call post()
...
```

👉 Method না থাকলে → **Method Not Allowed**

🔹 If you DON’T want this manual work?

Option 1️⃣ GenericAPIView

Auto handle অনেক কিছু

Option 2️⃣ ModelViewSet

সব CRUD auto

🔹 Interview-ready one liner

> APIView এ HTTP operation অনুযায়ী method define না করলে DRF `405 Method Not Allowed` return করে।

🔥 Bottom line

- APIView = **manual control**

- Method না লিখলে = operation বন্ধ
- Auto চাইলে = ViewSet

Good question 👍
APIView use করলে router কেমন হবে—এটা অনেকের confusion হয়।

🔹 Short answer

👉 APIView এর জন্য DRF router use হয় না ❌
👉 URL pattern manually লিখতে হয় ✅

🔹 APIView এর URL config (correct way)

Example: `urls.py`

```
from django.urls import path
from .views import EmployeeAPIView

urlpatterns = [
    path("employees/", EmployeeAPIView.as_view()),
    path("employees/<int:pk>/", EmployeeAPIView.as_view()),
]
```

👉 এখানে:

- `/employees/` → GET (list), POST (create)
- `/employees/1/` → GET (detail), PUT, PATCH, DELETE

🔹 APIView এর complete flow

```
class EmployeeAPIView(APIView):

    def get(self, request, pk=None):
        if pk:
            emp = Employee.objects.get(id=pk)
            serializer = EmployeeSerializer(emp)
        else:
            emp = Employee.objects.all()
```

```
            serializer = EmployeeSerializer(emp, many=True)
        return Response(serializer.data)
```

👉 **same APIView class**
👉 **router নেই**
👉 **URL দিয়ে control**

🔹 Common mistake ❌

```
router = DefaultRouter()
router.register("employees", EmployeeAPIView)
```

❌ এটা কাজ করবে না
👉 Router শুধু **ViewSet** এর জন্য

🔹 Router কাদের জন্য?

Class	Router
APIView	❌ No
GenericAPIView	❌ No
ViewSet	✅ Yes
ModelViewSet	✅ Yes

🔹 ViewSet হলে router কেমন হয়? (compare)

```
router = DefaultRouter()
router.register("employees", EmployeeViewSet)

urlpatterns = router.urls
```

👉 Automatically:

- GET /employees/
- POST /employees/
- GET /employees/1/
- PUT / PATCH / DELETE /employees/1/

🧠 Interview one-liner

> APIView এ router ব্যবহার হয় না, manually URL define করতে হয়।

🔥 Summary

- APIView → `path()` দিয়ে URL
- Router → শুধু ViewSet
- Control বেশি → APIView
- Code কম → ViewSet

চাও তো আমি

- **APIView vs ViewSet URL diagram**
- **Best practice project structure**
- **Real REST API mapping table**
  দেখাতে পারি 😊

Nice 👍

DRF-এ কী কী ধরনের ViewSet আছে — একদম clear করে দিচ্ছি।

🔹 Django REST Framework ViewSet এর ধরন

DRF-এ মোট **৪টা main ViewSet class** আছে 👇

1️⃣ `ViewSet`

👉 **সবচেয়ে basic**

```
from rest_framework.viewsets import ViewSet

class EmployeeViewSet(ViewSet):

    def list(self, request):
```

```
        ...

    def retrieve(self, request, pk=None):
        ...

    def create(self, request):
        ...

    def update(self, request, pk=None):
        ...

    def partial_update(self, request, pk=None):
        ...

    def destroy(self, request, pk=None):
        ...
```

📌 সব নিজে লিখতে হয়
📌 Full control
📌 DB auto handle করে না

2️⃣ `GenericViewSet`

👉 `ViewSet` + `GenericAPIView`

```
from rest_framework.viewsets import GenericViewSet

class EmployeeViewSet(GenericViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
```

📌 queryset + serializer support
📌 Usually mixin এর সাথে use হয়

3️⃣ `ReadOnlyModelViewSet`

👉 Only **GET allowed**

```
from rest_framework.viewsets import ReadOnlyModelViewSet

class EmployeeViewSet(ReadOnlyModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
```

Auto methods:

- list
- retrieve

❌ create / update / delete নাই

4️⃣ `ModelViewSet` ⭐ (সবচেয়ে বেশি use হয়)

```
from rest_framework.viewsets import ModelViewSet

class EmployeeViewSet(ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
```

Auto methods:

- list (GET)
- retrieve (GET)
- create (POST)
- update (PUT)
- partial_update (PATCH)
- destroy (DELETE)

📌 Production favorite
📌 Router required

🔹 ViewSet method mapping (important)

Method	URL	HTTP
list	`/employees/`	GET
retrieve	`/employees/1/`	GET
create	`/employees/`	POST
update	`/employees/1/`	PUT
partial_update	`/employees/1/`	PATCH
destroy	`/employees/1/`	DELETE

🔹 Router example

```
router = DefaultRouter()
router.register("employees", EmployeeViewSet)

urlpatterns = router.urls
```

🧠 Interview short answer

> DRF-এ ViewSet চার ধরনের: ViewSet, GenericViewSet, ReadOnlyModelViewSet, ModelViewSet।

🔥 Which one use করবে?

Situation	Best choice
Custom logic	ViewSet
Mixins	GenericViewSet
Read-only API	ReadOnlyModelViewSet
Full CRUD	ModelViewSet

Short answer: **না—ViewSet use করলে সব method override করতে হয় না।**
এখন step-by-step clear করি 👇

🔹 1️⃣ ModelViewSet হলে (সবচেয়ে common)

```
class EmployeeViewSet(ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
```

👉 এখানে তুমি **কোনো method override না করলেও** নিচের সব operation কাজ করবে:

HTTP	Method (auto)
GET (list)	`list()`

HTTP	Method (auto)
GET (detail)	`retrieve()`
POST	`create()`
PUT	`update()`
PATCH	`partial_update()`
DELETE	`destroy()`

✅ **Override করার দরকার নেই** (by default)

🔹 2️⃣ Override কখন করবে?

👉 যখন **custom logic** দরকার হয়

Example: `create()` override

```
class EmployeeViewSet(ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer

    def create(self, request, *args, **kwargs):
        print("Custom create logic")
        return super().create(request, *args, **kwargs)
```

📌 শুধু দরকার হলে override করো

🔹 3️⃣ ReadOnlyModelViewSet হলে

```
class EmployeeViewSet(ReadOnlyModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
```

👉 Auto methods:

- `list()`
- `retrieve()`

❌ create / update / delete নাই
❌ Override না করলে POST দিলে 405 আসবে

🔹 4️⃣ ViewSet (basic) হলে

```
class EmployeeViewSet(ViewSet):
    def list(self, request):
        ...

    def retrieve(self, request, pk=None):
        ...
```

👉 এখানে **সব method নিজে লিখতে হয়**
👉 না লিখলে → 405 Method Not Allowed

🔹 5️⃣ Summary table (most important)

ViewSet type	Override দরকার?
ViewSet	✅ সব
GenericViewSet	🔶 mixin অনুযায়ী
ReadOnlyModelViewSet	❌ না
ModelViewSet	❌ না

🧠 Interview one-liner

> ModelViewSet use করলে CRUD method override করা লাগে না, শুধু custom logic থাকলে override করা হয়।

🔥 Bottom line

- Fast API → `ModelViewSet`
- Custom control → override
- Full manual → `ViewSet`

চাও তো আমি

- **APIView vs ViewSet comparison**
- **Custom permission / filter example**

- **Override best practices**
  দেখাতে পারি 😊

Perfect 👍
এখন **একই সব method use করে**
👉 **Basic ViewSet**
👉 **ModelViewSet**
দুটোর **full working example** দিচ্ছি — side-by-side বুঝতে সুবিধা হবে।

🔹 1️⃣ Basic ViewSet (সব method MANUAL)

```
from rest_framework.viewsets import ViewSet
from rest_framework.response import Response
from rest_framework import status
from .models import Employee
from .serializers import EmployeeSerializer

class EmployeeViewSet(ViewSet):

    def list(self, request):
        qs = Employee.objects.all()
        serializer = EmployeeSerializer(qs, many=True)
        return Response(serializer.data)

    def retrieve(self, request, pk=None):
        emp = Employee.objects.get(id=pk)
        serializer = EmployeeSerializer(emp)
        return Response(serializer.data)

    def create(self, request):
        serializer = EmployeeSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        serializer.save()
        return Response(serializer.data, status=status.HTTP_201_CREATED)

    def update(self, request, pk=None):
        emp = Employee.objects.get(id=pk)
        serializer = EmployeeSerializer(emp, data=request.data)
        serializer.is_valid(raise_exception=True)
        serializer.save()
        return Response(serializer.data)

    def partial_update(self, request, pk=None):
        emp = Employee.objects.get(id=pk)
```

```
        serializer = EmployeeSerializer(
            emp, data=request.data, partial=True
        )
        serializer.is_valid(raise_exception=True)
        serializer.save()
        return Response(serializer.data)

    def destroy(self, request, pk=None):
        emp = Employee.objects.get(id=pk)
        emp.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
```

👉 এখানে:

- method নাম **same**
- সব logic **নিজে লিখতে হয়েছে**
- DB + serializer সব manual

🔹 2️⃣ ModelViewSet (সব method AUTO)

```
from rest_framework.viewsets import ModelViewSet
from .models import Employee
from .serializers import EmployeeSerializer

class EmployeeModelViewSet(ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
```

👉 এই ৩ লাইনে DRF auto করে দেয় 👇

Method	Auto handle
`list()`	✅
`retrieve()`	✅
`create()`	✅
`update()`	✅
`partial_update()`	✅
`destroy()`	✅

🔹 3️⃣ Router (দুটোর জন্য SAME)

```python
from rest_framework.routers import DefaultRouter
from .views import EmployeeViewSet, EmployeeModelViewSet

router = DefaultRouter()
router.register("employees", EmployeeViewSet)
# অথবা
# router.register("employees", EmployeeModelViewSet)

urlpatterns = router.urls
```

🔹 4️⃣ URL → Method mapping (important)

HTTP	URL	Method
GET	/employees/	`list()`
GET	/employees/1/	`retrieve()`
POST	/employees/	`create()`
PUT	/employees/1/	`update()`
PATCH	/employees/1/	`partial_update()`
DELETE	/employees/1/	`destroy()`

🔹 5️⃣ Core difference (1 লাইনে)

- **ViewSet** → method নাম same, code তুমি লেখো
- **ModelViewSet** → method নাম same, code DRF লেখে 😄

🧠 Interview final answer

> Basic ViewSet এবং ModelViewSet–এর method নাম একই (list, retrieve, create, update, partial_update, destroy), পার্থক্য হলো implementation—ViewSet এ manual, ModelViewSet এ automatic।

চাও তো next step এ আমি দেখাতে পারি:

- **একটা method override করলে ভিতরে কী ঘটে**

- **APIView → ViewSet conversion**
- **Real production best practice**

বললেই হলো 👌

ঠিক আছে 🙂
এখন আমি **ModelViewSet এ "all method"** একদম clearly দেখাচ্ছি — যা DRF auto provide করে, এবং কোন URL / HTTP method এর সাথে map হয়।

🔹 ModelViewSet – All methods

`ModelViewSet` automatically handle করে নিচের ৬টি main methods:

Method name	HTTP method	URL (Router এর সাথে)	Purpose
`list`	GET	/employees/	সব objects list করা
`retrieve`	GET	/employees/{pk}/	single object detail
`create`	POST	/employees/	নতুন object তৈরি
`update`	PUT	/employees/{pk}/	পুরো object update
`partial_update`	PATCH	/employees/{pk}/	শুধু আসা field update
`destroy`	DELETE	/employees/{pk}/	object delete করা

🔹 Example – ModelViewSet

```
from rest_framework.viewsets import ModelViewSet
from .models import Employee
from .serializers import EmployeeSerializer

class EmployeeModelViewSet(ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
```

কোনো method override না করলে DRF এই ৬টি method auto handle করবে।

🔹 Router example

```python
from rest_framework.routers import DefaultRouter
from .views import EmployeeModelViewSet

router = DefaultRouter()
router.register("employees", EmployeeModelViewSet)

urlpatterns = router.urls
```

এখন automatically:

- GET /employees/ → list
- GET /employees/1/ → retrieve
- POST /employees/ → create
- PUT /employees/1/ → update
- PATCH /employees/1/ → partial_update
- DELETE /employees/1/ → destroy

🔹 Key points

1. Method **name same** হয় (list, retrieve, create, update, partial_update, destroy)
2. Implementation **DRF handle করে**, তুমি override না করলেও চলে
3. Override করার দরকার শুধু যখন **custom logic** লাগে

ঠিক আছে 🙂
এখন আমি দেখাব **ModelViewSet এ সব method override করে কিভাবে লেখা হয়**, যাতে তুমি স্পষ্টভাবে বুঝতে পারো—DRF auto দেয়, কিন্তু আমরা চাইলে override করতে পারি।

🔹 Full ModelViewSet with all methods overridden

```python
from rest_framework.viewsets import ModelViewSet
from rest_framework.response import Response
from rest_framework import status
from .models import Employee
from .serializers import EmployeeSerializer

class EmployeeModelViewSet(ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
```

```
    # 1️⃣ List all employees
    def list(self, request, *args, **kwargs):
        queryset = self.get_queryset()
        serializer = self.get_serializer(queryset, many=True)
        return Response(serializer.data)

    # 2️⃣ Retrieve single employee
    def retrieve(self, request, pk=None, *args, **kwargs):
        emp = self.get_object()
        serializer = self.get_serializer(emp)
        return Response(serializer.data)

    # 3️⃣ Create new employee
    def create(self, request, *args, **kwargs):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        self.perform_create(serializer)
        return Response(serializer.data, status=status.HTTP_201_CREATED)

    # 4️⃣ Update full employee object (PUT)
    def update(self, request, pk=None, *args, **kwargs):
        emp = self.get_object()
        serializer = self.get_serializer(emp, data=request.data)
        serializer.is_valid(raise_exception=True)
        self.perform_update(serializer)
        return Response(serializer.data)

    # 5️⃣ Partial update (PATCH)
    def partial_update(self, request, pk=None, *args, **kwargs):
        emp = self.get_object()
        serializer = self.get_serializer(emp, data=request.data,
partial=True)
        serializer.is_valid(raise_exception=True)
        self.perform_update(serializer)
        return Response(serializer.data)

    # 6️⃣ Delete employee
    def destroy(self, request, pk=None, *args, **kwargs):
        emp = self.get_object()
        self.perform_destroy(emp)
        return Response(status=status.HTTP_204_NO_CONTENT)
```

🔹 Key points

1. **Method names same**: `list, retrieve, create, update, partial_update, destroy`

2. **Default DRF behavior**: তুমি override না করলে ModelViewSet এগুলো auto provide করে।
3. **Override দরকার তখন**: যদি তুমি extra logic চালাতে চাও যেমন logging, permission, custom response, validation।
4. **Helper methods DRF**: `get_object()`, `get_queryset()`, `get_serializer()` → override না করলেও safe।

🔹 Router example

```
from rest_framework.routers import DefaultRouter
from .views import EmployeeModelViewSet

router = DefaultRouter()
router.register("employees", EmployeeModelViewSet)

urlpatterns = router.urls
```

এখন এই mapping হবে:

HTTP	URL	Method
GET	/employees/	list
GET	/employees/1/	retrieve
POST	/employees/	create
PUT	/employees/1/	update
PATCH	/employees/1/	partial_update
DELETE	/employees/1/	destroy

💡 **Tip:** তুমি সব method override করে দেখলে বুঝবে কোথায় DRF auto কাজ করে আর কোথায় তুমি custom logic add করতে পারো।

যদি চাও আমি এবার **APIView → ModelViewSet conversion full example** দেখাই যেখানে সব method একসাথে কাজ করছে + router mapping diagram সহ।
এটা বুঝতে অনেক সহজ হবে।

চাও কি আমি বানাই?

ঠিক আছে! 😎

আমি এখন তোমার জন্য **full `ModelViewSet` example** বানাবো, যেখানে থাকবে:

- সব CRUD method: `list`, `retrieve`, `create`, `update`, `partial_update`, `destroy`
- **Custom query → operation → update → custom response**
- Response-এ কাস্টম data দেখানো

🔹 Full Example – EmployeeModelViewSet

```
from rest_framework.viewsets import ModelViewSet
from rest_framework.decorators import action
from rest_framework.response import Response
from rest_framework import status
from django.db.models import F
from .models import Employee
from .serializers import EmployeeSerializer
import requests  # external API optional

class EmployeeModelViewSet(ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer

    # ----------------------
    # 1️⃣ All CRUD methods (override optional)
    # ----------------------

    def list(self, request, *args, **kwargs):
        queryset = self.get_queryset()
        serializer = self.get_serializer(queryset, many=True)
        return Response(serializer.data)

    def retrieve(self, request, pk=None, *args, **kwargs):
        emp = self.get_object()
        serializer = self.get_serializer(emp)
        return Response(serializer.data)

    def create(self, request, *args, **kwargs):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        emp = serializer.save()
        return Response(self.get_serializer(emp).data,
status=status.HTTP_201_CREATED)

    def update(self, request, pk=None, *args, **kwargs):
        emp = self.get_object()
        serializer = self.get_serializer(emp, data=request.data)
        serializer.is_valid(raise_exception=True)
```

```
        emp = serializer.save()
        return Response(self.get_serializer(emp).data)

    def partial_update(self, request, pk=None, *args, **kwargs):
        emp = self.get_object()
        serializer = self.get_serializer(emp, data=request.data,
partial=True)
        serializer.is_valid(raise_exception=True)
        emp = serializer.save()
        return Response(self.get_serializer(emp).data)

    def destroy(self, request, pk=None, *args, **kwargs):
        emp = self.get_object()
        emp.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

    # ----------------------
    # 2️⃣ Custom query + operation
    # ----------------------
    @action(detail=False, methods=['POST'])
    def apply_bonus(self, request):
        """
        Custom operation:
        - Query employees with salary < 50000
        - Add bonus
        - Update DB
        - Return custom response
        """
        # Example: custom bonus from request or default 5000
        bonus_amount = request.data.get("bonus", 5000)

        # Query employees
        employees = Employee.objects.filter(salary__lt=50000)

        result = []
        for emp in employees:
            # Optional: external API for bonus
            # external =
requests.get(f"https://api.example.com/bonus/{emp.id}").json()
            # bonus = external.get("bonus", bonus_amount)

            emp.salary += bonus_amount
            emp.save()

            # Prepare custom response
            result.append({
                "id": emp.id,
                "name": emp.name,
                "bonus_added": bonus_amount,
                "total_salary": emp.salary
```

```
            })

        return Response({
            "message": f"Bonus applied to {len(result)} employees",
            "employees": result
        }, status=status.HTTP_200_OK)
```

🔹 Router setup

```
from rest_framework.routers import DefaultRouter
from .views import EmployeeModelViewSet

router = DefaultRouter()
router.register("employees", EmployeeModelViewSet)

urlpatterns = router.urls
```

🔹 URL → Method mapping

HTTP	URL	ModelViewSet Method	Purpose
GET	/employees/	list	All employees
GET	/employees/{pk}/	retrieve	Single employee
POST	/employees/	create	Create employee
PUT	/employees/{pk}/	update	Full update
PATCH	/employees/{pk}/	partial_update	Partial update
DELETE	/employees/{pk}/	destroy	Delete employee
POST	/employees/apply_bonus/	apply_bonus	Custom query + operation

---

🔹 Features of this example

1. ✅ **All CRUD methods** override for clarity
2. ✅ **Custom query operation** via `@action(detail=False)`
3. ✅ **Database update** inside custom method

4. ✅ **Custom response** with original + updated info
5. ✅ Optional external operation (commented `requests`)

💡 **Tip:**

- `@action(detail=False)` → collection level action (all employees)
- `@action(detail=True)` → object level action (single employee)

হ্যাঁ ঠিক! 😄

`@action(detail=True)` ব্যবহার করলে **DRF automatically ধরে নেয় এটা single object action**, তাই **URL-এ object-এর `id` দিতে হয়।**

🔹 Explanation

```
from rest_framework.decorators import action
from rest_framework.response import Response
from rest_framework import status

class EmployeeModelViewSet(ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer

    @action(detail=True, methods=['POST'])
    def give_bonus(self, request, pk=None):
        emp = self.get_object()  # automatically fetches by pk from URL
        bonus = request.data.get("bonus", 1000)
        emp.salary += bonus
        emp.save()
        return Response({
            "id": emp.id,
            "name": emp.name,
            "bonus_added": bonus,
            "total_salary": emp.salary
        }, status=status.HTTP_200_OK)
```

🔹 URL Example

Router automatically create করবে:

```
POST /employees/1/give_bonus/
```

- এখানে `1` হলো **pk / id**
- `self.get_object()` method সেই id অনুযায়ী object fetch করবে
- তুমি URL-এ অন্য id দিতে পারো যেমন `/employees/5/give_bonus/`

🔹 Key points

Parameter	Meaning
`detail=True`	Single object action (need `pk`)
`detail=False`	Collection action (all objects, no `pk`)
`self.get_object()`	DRF automatically fetch object by pk

💡 **Tip:**

- Single object action = `detail=True` → URL must contain pk
- Collection action = `detail=False` → URL does **not** need pk

ঠিক আছে 🙂
এখন আমি step-by-step বুঝাচ্ছি **Nested Router** কীভাবে কাজ করে, এবং DRF-এ এটা কখন লাগে।

🔹 1️⃣ Nested Router কী?

- Normal router:

```
/employees/          → list, create
/employees/1/        → retrieve, update, delete
```

- Nested router:
  যখন একটা resource **অন্য resource এর under** থাকে।

Example:

- Employee → multiple Tasks

- Task → belongs to Employee

```
/employees/1/tasks/        → list all tasks of employee 1
/employees/1/tasks/5/      → retrieve task 5 of employee 1
```

এখানে `tasks` nested under `employees`

🔹 2️⃣ Install dependency

DRF default router nested support দেয় না। সাধারণত `drf-nested-routers` ব্যবহার হয়:

```
pip install drf-nested-routers
```

🔹 3️⃣ Example models

```
from django.db import models

class Employee(models.Model):
    name = models.CharField(max_length=100)
    salary = models.FloatField()

class Task(models.Model):
    employee = models.ForeignKey(Employee, related_name="tasks",
on_delete=models.CASCADE)
    title = models.CharField(max_length=200)
    completed = models.BooleanField(default=False)
```

🔹 4️⃣ Serializers

```
from rest_framework import serializers
from .models import Employee, Task

class TaskSerializer(serializers.ModelSerializer):
    class Meta:
        model = Task
        fields = "__all__"

class EmployeeSerializer(serializers.ModelSerializer):
    tasks = TaskSerializer(many=True, read_only=True)  # nested display
```

```
    class Meta:
        model = Employee
        fields = "__all__"
```

🔹 5️⃣ ViewSets

```
from rest_framework.viewsets import ModelViewSet
from .models import Employee, Task
from .serializers import EmployeeSerializer, TaskSerializer

class EmployeeViewSet(ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer

class TaskViewSet(ModelViewSet):
    serializer_class = TaskSerializer

    def get_queryset(self):
        employee_id = self.kwargs.get("employee_pk")  # nested router pass
করে
        return Task.objects.filter(employee_id=employee_id)

    def perform_create(self, serializer):
        employee_id = self.kwargs.get("employee_pk")
        serializer.save(employee_id=employee_id)
```

🔹 6️⃣ Nested Router setup

```
from rest_framework_nested import routers
from .views import EmployeeViewSet, TaskViewSet

router = routers.DefaultRouter()
router.register(r'employees', EmployeeViewSet, basename='employee')

# nested router
employees_router = routers.NestedDefaultRouter(router, r'employees',
lookup='employee')
employees_router.register(r'tasks', TaskViewSet, basename='employee-
tasks')

urlpatterns = router.urls + employees_router.urls
```

🔹 7️⃣ URL mapping

URL	Method	Purpose
/employees/	GET	list employees
/employees/1/	GET	retrieve employee 1
/employees/1/tasks/	GET	list tasks of employee 1
/employees/1/tasks/5/	GET	retrieve task 5 of employee 1
/employees/1/tasks/	POST	create task under employee 1

🔹 8️⃣ Key points

1. `lookup='employee'` → nested pk name, automatically available in `kwargs`
2. Nested ViewSet → `get_queryset()` override করে parent pk ব্যবহার করে query
3. `perform_create()` override করে parent relation save করা

💡 **Tip:**

- Collection-level nested action: `detail=False`
- Single object nested action: `detail=True`
- DRF nested router → DRY, clean URL structure

হ্যাঁ! 😄 তুমি যদি **multiple nested routers** ব্যবহার করো, যেমন:

```
/employees/{employee_id}/tasks/{task_id}/subtasks/{subtask_id}/
```

তাহলে DRF automatically **kwargs** এর মাধ্যমে সব parent এবং current ids দেয়। তুমি সেগুলো **ViewSet থেকে access করতে পারো**।

🔹 Example Scenario

- Employee → Task → SubTask

```
# models.py
class Employee(models.Model):
```

```
    name = models.CharField(max_length=100)

class Task(models.Model):
    employee = models.ForeignKey(Employee, related_name="tasks",
on_delete=models.CASCADE)
    title = models.CharField(max_length=100)

class SubTask(models.Model):
    task = models.ForeignKey(Task, related_name="subtasks",
on_delete=models.CASCADE)
    title = models.CharField(max_length=100)
```

🔹 Serializers

```
class SubTaskSerializer(serializers.ModelSerializer):
    class Meta:
        model = SubTask
        fields = "__all__"

class TaskSerializer(serializers.ModelSerializer):
    subtasks = SubTaskSerializer(many=True, read_only=True)
    class Meta:
        model = Task
        fields = "__all__"

class EmployeeSerializer(serializers.ModelSerializer):
    tasks = TaskSerializer(many=True, read_only=True)
    class Meta:
        model = Employee
        fields = "__all__"
```

🔹 ViewSet – access all ids

```
from rest_framework.viewsets import ModelViewSet
from .models import SubTask
from .serializers import SubTaskSerializer

class SubTaskViewSet(ModelViewSet):
    serializer_class = SubTaskSerializer

    def get_queryset(self):
        employee_id = self.kwargs.get("employee_pk")
        task_id = self.kwargs.get("task_pk")
```

```
        return SubTask.objects.filter(task_id=task_id,
task__employee_id=employee_id)

    def perform_create(self, serializer):
        task_id = self.kwargs.get("task_pk")
        serializer.save(task_id=task_id)
```

🔹 Nested Router setup

```
from rest_framework_nested import routers
from .views import EmployeeViewSet, TaskViewSet, SubTaskViewSet

router = routers.DefaultRouter()
router.register(r'employees', EmployeeViewSet, basename='employee')

tasks_router = routers.NestedDefaultRouter(router, r'employees',
lookup='employee')
tasks_router.register(r'tasks', TaskViewSet, basename='employee-tasks')

subtasks_router = routers.NestedDefaultRouter(tasks_router, r'tasks',
lookup='task')
subtasks_router.register(r'subtasks', SubTaskViewSet, basename='task-
subtasks')

urlpatterns = router.urls + tasks_router.urls + subtasks_router.urls
```

🔹 URL → kwargs mapping

URL	kwargs in ViewSet
/employees/1/tasks/2/subtasks/	`employee_pk=1``task_pk=2`
/employees/1/tasks/2/subtasks/5/	`employee_pk=1``task_pk=2``pk=5`

- `self.kwargs['employee_pk']` → parent Employee ID
- `self.kwargs['task_pk']` → parent Task ID
- `self.kwargs['pk']` → current SubTask ID

💡 **Tip:**

- Multiple nested levels → সব parent pk `kwargs` হিসেবে available
- `get_queryset()` → filter করতে parent-child relation enforce করা যায়
- `perform_create()` → save করার সময় parent relation set করা যায়

ঠিক আছে 🙂 তুমি বলতে চাও যে Nested Router-এ প্রতিটি Employee এর জন্য তার **own pk** থাকবে এবং তার নিচের resource (যেমন Task, SubTask) **relative** হবে, যেন URL structure হয় কিছুটা এভাবে:

```
/employees/{employee_pk}/tasks/{task_pk}/subtasks/{subtask_pk}/
```

এখানে:

- **employee_pk** → Employee model-এর primary key
- **task_pk** → Task model-এর primary key
- **subtask_pk** → SubTask model-এর primary key (current object, optional detail view)

🔹 Nested Router behavior

1. **DefaultRouter** → Employees collection handle করে

```
/employees/          → list, create
/employees/1/        → retrieve, update, delete
```

2. **NestedDefaultRouter** → Employee এর under Task handle করে

```
/employees/1/tasks/       → list tasks of employee 1
/employees/1/tasks/2/     → retrieve task 2 of employee 1
```

3. **Next level nested** → SubTask handle করে

```
/employees/1/tasks/2/subtasks/        → list subtasks of task 2 of
employee 1
/employees/1/tasks/2/subtasks/5/      → retrieve subtask 5
```

🔹 kwargs mapping in ViewSet

Nested router automatically passes all parent PKs in `kwargs`:

```python
# SubTaskViewSet
def get_queryset(self):
    employee_id = self.kwargs.get("employee_pk")  # parent employee
```

```
    task_id = self.kwargs.get("task_pk")          # parent task
    return SubTask.objects.filter(task_id=task_id,
task__employee_id=employee_id)

def perform_create(self, serializer):
    task_id = self.kwargs.get("task_pk")
    serializer.save(task_id=task_id)
```

- **kwargs['employee_pk']** → Employee ID
- **kwargs['task_pk']** → Task ID
- **kwargs['pk']** → SubTask ID (যদি detail view হয়)

🔹 Key points

1. প্রতিটি resource (Employee → Task → SubTask) তার parent pk অনুসারে URL হবে
2. `kwargs` automatically available হবে nested router দ্বারা
3. CRUD + custom operations সব nested level এ same logic অনুযায়ী handle করা যাবে

💡 **Tip:**

- Employee collection → `/employees/`
- Single employee → `/employees/{employee_pk}/`
- Nested resources → `/employees/{employee_pk}/tasks/...`

ঠিক আছে 🙂
তুমি জানতে চাচ্ছো `get_queryset()` **method কখন এবং কেন ব্যবহার করা হয়**। আমি step-by-step explain করছি।

🔹 1️⃣ `get_queryset()` কী?

- এটি **DRF ViewSet বা Generic View**-এর method
- এটি **queryset define / customize করার জন্য** ব্যবহার হয়
- DRF auto call করে যখন **list, retrieve, update, delete** operations করা হয়
- Return value **must be a queryset**

🔹 2️⃣ কখন ব্যবহার করা হয়?

1. Dynamic filtering / nested resources

যখন **queryset parent / request / kwargs অনুযায়ী change** করতে হবে।

Example – Nested Router

```
class TaskViewSet(ModelViewSet):
    serializer_class = TaskSerializer

    def get_queryset(self):
        # employee_pk nested router থেকে আসছে
        employee_id = self.kwargs.get("employee_pk")
        # শুধু ওই employee এর task filter করে return
        return Task.objects.filter(employee_id=employee_id)
```

> এখানে list / detail view এ auto filter হবে parent Employee অনুযায়ী

---

2. Request user অনুযায়ী data limit করা

```
class EmployeeViewSet(ModelViewSet):
    serializer_class = EmployeeSerializer

    def get_queryset(self):
        # যদি user শুধু নিজের data দেখতে পারে
        user = self.request.user
        return Employee.objects.filter(created_by=user)
```

---

3. Extra filtering / search / query params

```
class EmployeeViewSet(ModelViewSet):
    serializer_class = EmployeeSerializer

    def get_queryset(self):
        qs = Employee.objects.all()
        min_salary = self.request.query_params.get("min_salary")
        if min_salary:
            qs = qs.filter(salary__gte=min_salary)
        return qs
```

> HTTP: `/employees/?min_salary=50000` → filter employees salary >= 50000

🔹 3️⃣ কখন override করার দরকার নেই?

- যদি **fixed queryset** থাকে এবং কোনো dynamic filter দরকার না হয়

```
class EmployeeViewSet(ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
```

> DRF auto এই queryset ব্যবহার করবে

🔹 4️⃣ Key points

Feature	`queryset`	`get_queryset()`
Static queryset	✅	Optional
Dynamic filtering (kwargs, request.user, query_params)	❌	✅
Nested router support	❌	✅
Custom logic per request	❌	✅

💡 **Rule of thumb:**

- **Fixed queryset** → class attribute `queryset` ব্যবহার করো
- **Dynamic / request-specific / nested resource** → `get_queryset()` override করো

Day2